Video Script (about 3 minutes)

Record your screen (OBS, Loom or Zoom) with the app open. Speak naturally; you don't need to read word for word.

0:00 - Intro (15 s) "Hi, I'm Vishal. For Task 1, I built **ClinicFlow AI**, an AI-powered patient intake and appointment automation tool for GP clinics. I chose healthcare because Gokul Infocare runs GP clinics like Robina 7 Day Doctors, so I wanted to solve a real problem for that kind of business."

0:15 - The problem (20 s) "At a busy clinic, reception staff spend a lot of time on the phone asking patients what's wrong, deciding how urgent it is and choosing an appointment type. Then the doctor starts each consult with no context. And an urgent patient can end up waiting behind routine ones."

0:35 - Demo: routine/urgent patient (40 s) - *show the patient page* "The patient describes their symptoms in their own words. I'll use the 'High fever' example. The AI turns this into a structured note: urgency **Urgent**, a clinical summary for the doctor, key symptoms, the right appointment type and questions the doctor may want to ask. The patient gets a simple, friendly message. Because it's urgent, it only shows today's and tomorrow's slots. I pick a time and the booking is confirmed automatically."

1:15 - Demo: emergency (20 s) "Now the most important part, safety. If I type 'chest pain spreading to my left arm', the system flags an **Emergency**, tells the patient to call 000 and blocks online booking. This doesn't depend only on the AI: there's a rule-based safety layer that always overrides the AI for red flags like chest pain, stroke signs or suicidal thoughts."

1:35 - Demo: staff dashboard (20 s) - *open /admin* "Staff log in and see the queue **sorted by AI urgency**: emergencies first, then urgent, then routine. They can open any patient to see the AI summary and update the status."

1:55 - AI tools & prompt engineering (40 s) - *show PROMPT_DOCUMENTATION.md / app/prompts.py* "I used **Devin**, an AI software engineer, to plan the architecture and build and test the code, and **Google Gemini** inside the app (it can switch to OpenAI with one setting). My first prompt was too vague: the AI returned free text and sometimes tried to diagnose. In version 2 I forced a JSON format and said 'do not diagnose'. In version 3 I added clear definitions for each urgency level, a rule to choose the more urgent level when unsure, a few-shot example, and protection against prompt injection by wrapping the patient's text in tags. I check every prompt change with an evaluation script that runs 12 patient cases, and the key number is unsafe under-triage, which must be zero."

2:35 - Challenges (20 s) "Challenges: AI output isn't always valid JSON, so I validate it, retry once, and fall back to an offline engine so the app never breaks. I also fixed a time-zone bug where a 9 am Brisbane booking showed as 11 pm, and tuned the rules so a repeat prescription goes to telehealth instead of a long consult."

2:55 - Outcome (15 s) "The final result is a working, tested and Dockerised web app: 20 automated tests, CI, and it's deployed online. It saves reception time, gives doctors context before the consult, and gets emergencies the right help faster. Thank you!"